

CMPT 361 Final Project

Abdul Aldery, Saad Foda, JC Forrester-Clarke, Samantha Skiba, and Mohamad Taher

key_generator

Purpose of Key Generator

- Securely creates and manages cryptographic keys for both server and clients.
- Generates RSA key pairs for secure communication.
- Creates AES symmetric keys for data encryption.
- Manages user credentials with random password generation and secure hashing.

key_generator

RSA and AES:

RSA (Rivest-Shamir-Adleman): A public-key encryption method used for secure communication. It uses two keys, one for encryption (public) and one for decryption (private).

AES (Advanced Encryption Standard): A symmetric encryption algorithm that uses the same key for both encryption and decryption, providing a fast and secure way to protect data.

Server

User Authentication (RSA Encryption)

- Loads the server's RSA private key to decrypt client credentials.
- Authenticates users by decrypting their username and password.
- Credentials are stored in plaintext

Email Encryption & Decryption (AES Encryption)

- Emails are encrypted using AES in ECB mode
- Encrypts messages before sending and decrypts them upon retrieval.

Server

Inbox & Email Management

- Stores emails in user-specific inbox directories.
- Lists inbox contents (subject, sender, timestamp).
- Allows users to view specific emails after authentication.

Server

Client Session Handling

- Manages user sessions with authentication, command execution, and secure communication.
- Commands:
 - Send Email (Encrypt and store email).
 - View Inbox (Retrieve email summaries).
 - Read Email (Decrypt and display selected email).
 - Exit (Terminate session securely).

Server

Server Initialization & Connection Handling

- Listens for client connections on localhost:13000.
- Handles one client at a time

Client

User Authentication:

- Collects username and password, encrypts them using the server's public key, and sends them for authentication.
- If authentication fails, the connection is terminated.

Symmetric Key Retrieval:

- If authentication succeeds, the client retrieves a symmetric key (AES) from a file to encrypt further communication.
- If the key cannot be accessed, the program terminates.

Client

User Menu Options:

- After authentication, the program presents options for:
 - Sending an email
 - Viewing the inbox list
 - Viewing specific email content
 - Terminating the connection

Client

Email Sending (emailSend):

- Encrypts and sends emails, allowing the user to include recipients, a title, and optional file attachments.

Inbox Viewing (inbox, emailView):

- Requests and displays a decrypted inbox list from the server.
- Allows the user to select an email and view its decrypted content.

Client

Secure Communication:

- Uses AES encryption for messages after authentication.
- Ensures all transmitted data is encrypted before sending to the server.

Client

User Authentication:

- The client collects the user's username and password, encrypts them using the server's public key, and sends them to the server for authentication.

Symmetric Key Retrieval:

- After successful authentication, the client retrieves the symmetric key (used for AES encryption) from a file to ensure secure communication.

Server enhanced

Client Authentication:

- The server authenticates clients by decrypting their credentials using RSA. This ensures that only authorized clients can interact with the server. This is crucial for verifying the identity of the client and preventing unauthorized access.

Message Integrity:

- The server ensures the integrity of the messages it receives and sends using SHA-256 hashing. Each message is accompanied by a hash value to detect any tampering or corruption during transmission. If the hash doesn't match, the server identifies potential tampering.

Server enhanced

Secure Communication:

- Once authenticated, all communication between the client and server is encrypted using AES. This guarantees that sensitive information (such as email contents or credentials) remains private and secure from interception.

Email Management:

- The server handles the storing, retrieval, and viewing of emails. Emails are saved in user-specific inbox directories and are accessible to clients upon request. The emails are decrypted before being sent to the client, ensuring confidentiality.

Server enhanced

Session Management:

- The server manages client sessions, including the command loop (send, receive, view emails) and proper clean-up after the session ends. This ensures that each client's communication is handled in isolation and securely.

Client Enhanced

Menu Options:

- The client offers options to the user, such as sending an email, viewing the inbox, viewing email content, or terminating the connection.

Email Sending:

- The client allows the user to compose and send emails, encrypting the message with AES encryption before sending it to the server.

Client Enhanced

Inbox Viewing:

- The client can retrieve and display a list of emails from the server, decrypting the inbox list for the user.

Message Integrity Check:

- The program hashes each message with SHA-256 before encryption, ensuring the message's integrity by comparing the hash after decryption to detect any tampering.

Vulnerability with client and server program

- **MitM Attack:** A third party intercepts data transfers, pretending to be both participants (client and server).
- **Potential Risks:**
 - Altered data.
 - Malicious links injected.
 - Information manipulation or compromise.
- **Without safeguards**, the attack goes undetected, compromising confidentiality and integrity.

Improvements with client_enhanced and server_enhanced program

Hash Function (e.g., SHA-256) used for message integrity.

How it works:

- A digital fingerprint (hash) is generated for the sent message.
- The recipient also generates an internal hash for the received message.
- Both fingerprints are compared:
 - If they match, the message is considered unaltered.
 - If they don't match, the message is flagged as tampered.

Benefit: Ensures the message's integrity and verifies that it hasn't been altered during transmission.

Contributions

Abdul Aldery

- Key_Generator.py Analysis
- Presentation

Saad Foda

- Server.py and Server_enhanced.py Analysis

JC Forrester-Clarke

- Programmed key_generator.py, client.py, server.py, client_enhanced.py, server_enhanced.py

Samantha Skiba

- Client.py and Client_enhance.py Analysis
- Presentation

Mohamad Taher

- Testing of key_generator.py, client.py, server.py, client_enhanced.py and server_enhanced.py

Demo Time